Progetto Client-Server Multithreading
Forza quattro

Hetmanov Oleksandr, Benedettini Marco, Randazzo Kevin

Indice

1 Introduzione **2**

2 Regole di Gioco **2**

3 Protocollo di Comunicazione **2**
3.1 Formato Generale dei Messaggi 3
3.2 Fase di Connessione 3
3.3 Fase di Gioco 3
3.4 Rappresentazione della griglia 4
3.5 Terminazione 4
3.6 Riepilogo Comandi 4

4 Sequence Diagram **5**

5 Architettura del Server **6**
5.1 Componenti del Server 7
5.2 Gestione della Concorrenza 7

6 Architettura del Client **9**
6.1 Componenti del Client 9
6.2 Gestione del Multithreading Lato Client 9

7 Conformità ai Requisiti **10**

8 Conclusioni **11**

9 Strumenti utilizzati per i class diagram e sequence diagram **11**

1 Introduzione

Il presente progetto consiste nella realizzazione di un sistema Client-Server multithreading per l'implementazione del gioco Forza 4 in architettura distribuita.

- Ogni partita richiede esattamente due giocatori.
- Supporto alla gestione di N partite simultanee.
- Isolamento completo delle sessioni di gioco.
- Server centrale responsabile della gestione e sincronizzazione dello stato della partita.

Il piano di gioco è rappresentato da una matrice 6x7 (dimensione standard del gioco ufficiale).

Il server rappresenta l'unica autorità responsabile della validazione delle mosse e della coerenza dello stato della partita.

2 Regole di Gioco

Il sistema implementa le regole ufficiali di Forza 4:

- Prima dell'avvio della partita, entrambi i giocatori forniscono nome ed età
- Il turno iniziale viene assegnato dal server al giocatore più giovane
- In caso di parità di età, il turno iniziale viene determinato dal server in modo casuale
- I giocatori inseriscono alternativamente un disco nella colonna scelta
- Il disco occupa la posizione libera più bassa della colonna, simulando l'effetto della gravità
- Vince il giocatore che allinea quattro dischi consecutivi in una delle seguenti direzioni:
  - Orizzontale
  - Verticale
  - Diagonale
- Se la matrice è piena senza che si verifichi una condizione di vittoria, la partita termina in pareggio
- In caso di disconnessione di un giocatore, la vittoria viene assegnata automaticamente all'avversario.
- Sistema di timeout per evitare blocchi della partita

3 Protocollo di Comunicazione

Il protocollo applicativo è di tipo testuale su trasporto TCP. Ogni messaggio è codificato come stringa ASCII.

3.1 Formato Generale dei Messaggi

Ogni messaggio segue il formato:

`TIPO_MESSAGGIO;Parametro1;Parametro2;...`

La comunicazione è sincrona lato protocollo, ma asincrona lato implementativo grazie all'uso di thread dedicati.

3.2 Fase di Connessione

Client → Server

- `CONNECT;Nome;Eta`

Esempio:

`CONNECT;Alberto;18`

Il server inserisce il giocatore in una coda di attesa fino al raggiungimento di due giocatori compatibili.

Server → Client

- `WELCOME;PlayerID`
- `WAIT`
- `START;PlayerIniziale`

dove:

- `PlayerID` identifica il giocatore nella partita (1 o 2).
- `PlayerIniziale` indica quale giocatore effettua la prima mossa.

3.3 Fase di Gioco

Client → Server

- `MOVE;Colonna`

Esempio:

`MOVE;3`

dove `Colonna` è un intero compreso tra 0 e 6.
Il giocatore per collocare il disco ha bisogno solo del numero della colonna.

Server → Client

- `VALID;BoardState`
- `ERROR;Motivazione`
- `TURN`
- `WIN`
- `LOSE`
- `DRAW`
- `TIMEOUT`

3.4 Rappresentazione della griglia

La griglia (matrice) è rappresentata come stringa lineare di 42 caratteri (6 righe x 7 colonne):

- `.` = cella vuota
- `x` = disco Player1
- `o` = disco Player2

Esempio:

```
VALID;......................xo.................
```

3.5 Terminazione

Client → Server

- `QUIT`

Il server assegna la vittoria all'avversario e chiude la GameRoom.

In caso di disconnessione improvvisa:

- Il server attende un timeout di riconnessione.
- Se il client non si riconnette, viene inviato `WIN` all'avversario.

3.6 Riepilogo Comandi

Client → Server

- `CONNECT;Nome;Eta`
- `MOVE;Colonna`
- `QUIT`

Server → Client

- `WELCOME;NumeroGiocatore`
- `WAIT;WaitingForOpponent`
- `START;TurnoIniziale`
- `TURN`
- `VALID;MatriceAggiornata`
- `ERROR;Descrizione`
- `WIN;Motivazione`

- `LOSE;Motivazione`
- `DRAW`
- `TIMEOUT`
- `DISCONNECT`

Tutti i messaggi sono stringhe testuali separate dal punto e virogla.

4 Sequence Diagram

Il diagramma di sequenza rappresenta:

- Connessione dei client
- Inserimento nella coda di attesa
- Creazione della GameRoom
- Avvio partita
- Alternanza dei turni
- Validazione mossa
- Aggiornamento matrice
- Condizione di vittoria/pareggio
- Gestione disconnessione

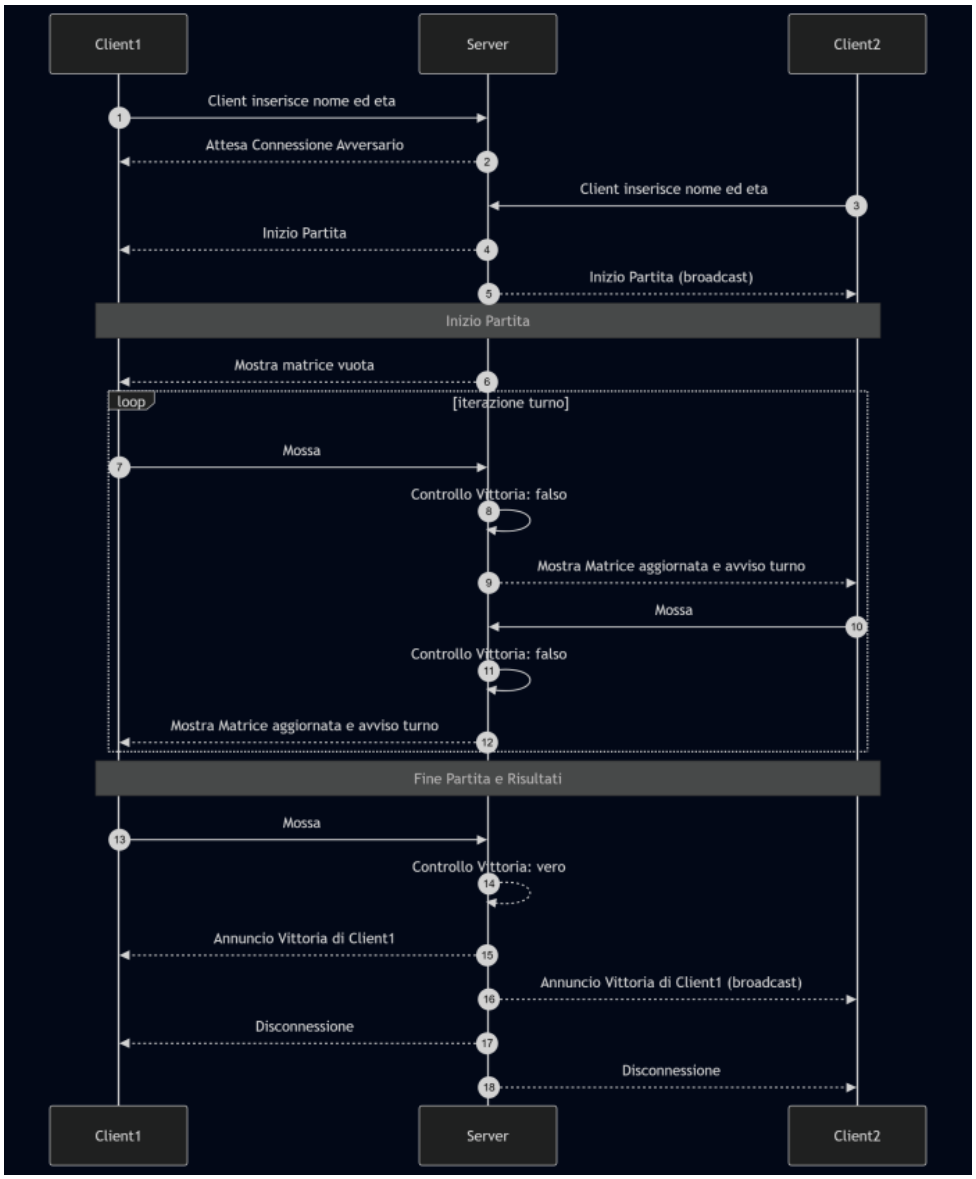

Figura 1: Diagramma Architettura del Server

5 Architettura del Server

Il server rappresenta il centro del sistema ed è strutturato con più thread, in modo da poter gestire contemporaneamente più partite.

5.1 Componenti del Server

Componente	Tipo	Responsabilità	Motivazione
Server	Classe	Avvio e accetta connessioni.	Punto d'ingresso, separa rete/logica.
ClientHandler	Runnable	I/O e comandi del client.	Isola la comunicazione per client.
GameManager	Classe	Coda e pairing giocatori.	Accoppiamento centralizzato.
GameRoom	Classe	Turni e stato partita.	Incapsula una partita.
TurnTimer	Classe	Timeout del turno.	Timer separato dalla logica.
MessageParser	Classe	Stringa → comando.	Parsing uniforme in ingresso.
MessageFormatter	Classe	Evento → stringa.	Serializzazione uniforme in uscita.
Board	Classe	Griglia 6x7, mosse, vittoria.	Logica scacchiera isolata.
Player	Classe	Dati giocatore.	Modello di dominio.
PlayerSession	Classe	Player + handler.	Riferimento univoco al client.
PlayerQueue	Classe	Coda giocatori (sync).	Struttura thread-safe.
GameQueue	Classe	Elenco partite attive.	Gestione sicura delle GameRoom.
GameState	Enum	Stati partita.	Ciclo di vita esplicito.
Token	Enum	Pedine X/O.	Evita valori magici.
ClientCommand	Interfaccia	Contratto dei comandi.	Polimorfismo lato server.
ServerEvent	Interfaccia	Contratto degli eventi.	Invio eventi indipendente.

5.2 Gestione della Concorrenza

Il server utilizza:

- Un thread principale per accettare connessioni.
- Un ClientHandler dedicato per ogni client.
- Una GameRoom indipendente per ogni partita.

Le operazioni critiche (inserimento disco, cambio turno, controllo vittoria) sono dichiarate `synchronized` per evitare race condition.

Ogni GameRoom mantiene il proprio stato interno, garantendo isolamento completo tra partite simultanee.

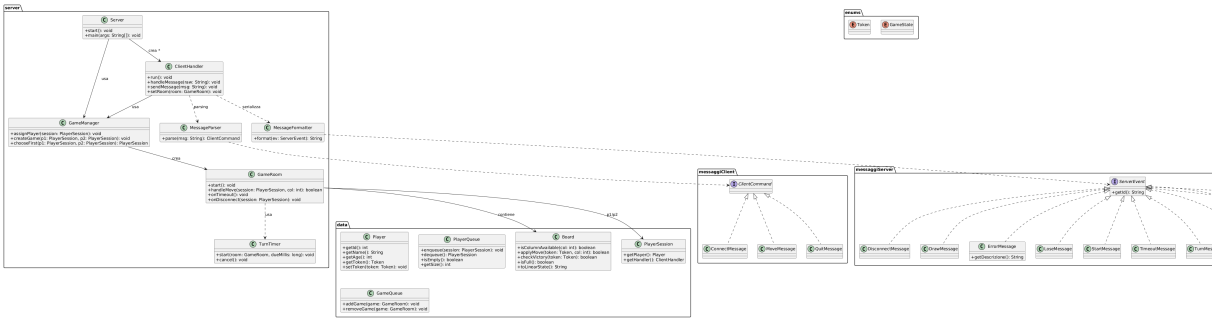

Figura 2: Diagramma Architettura del Server

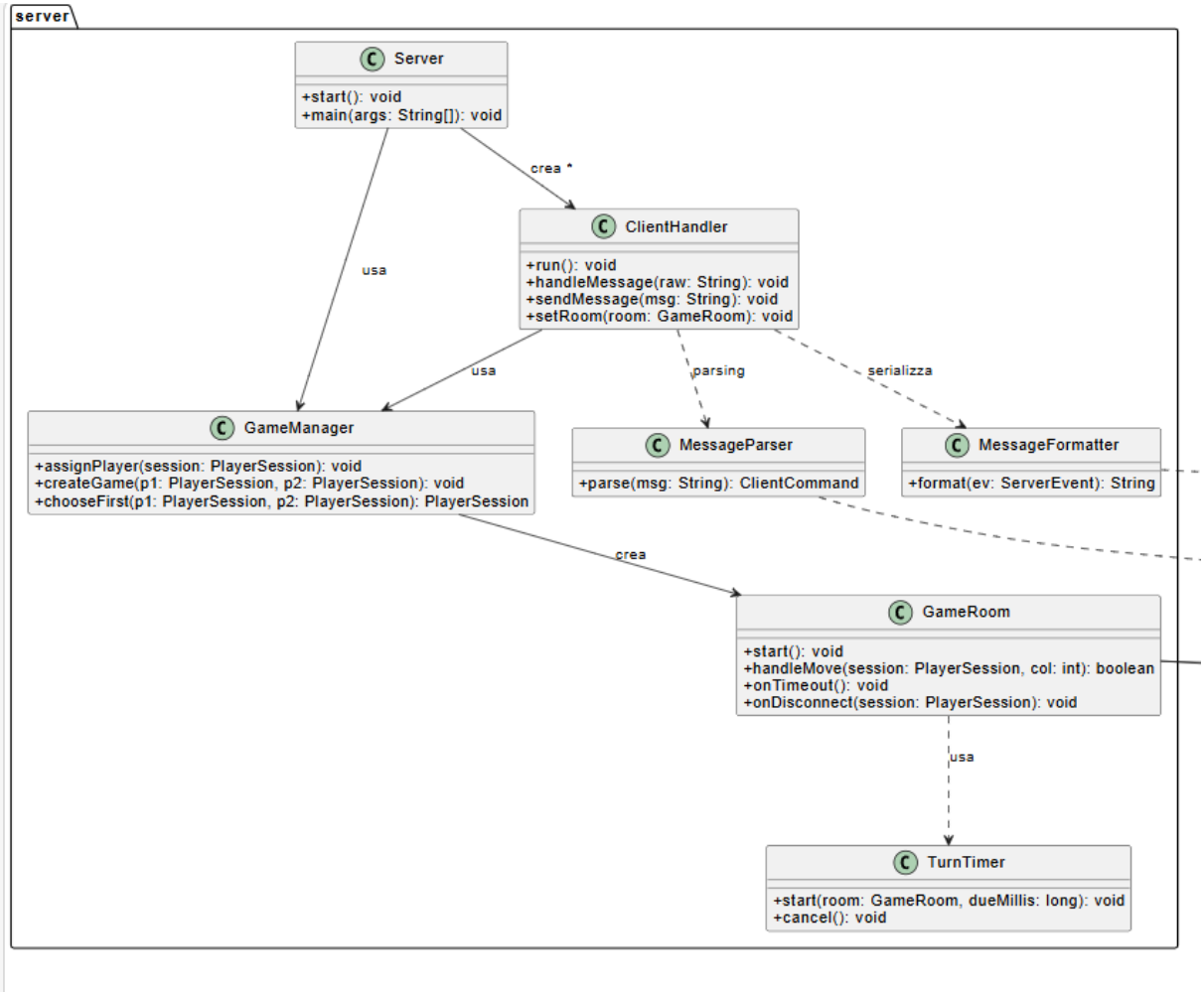

Figura 3: Diagramma Architettura del Server

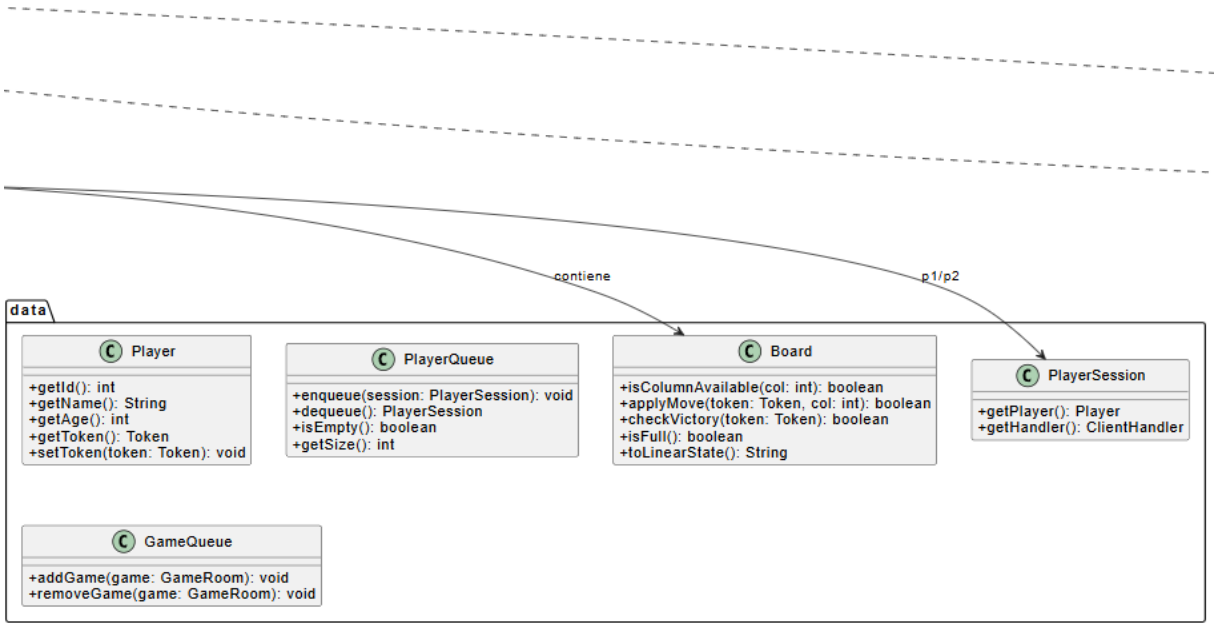

Figura 4: Diagramma Architettura del Server

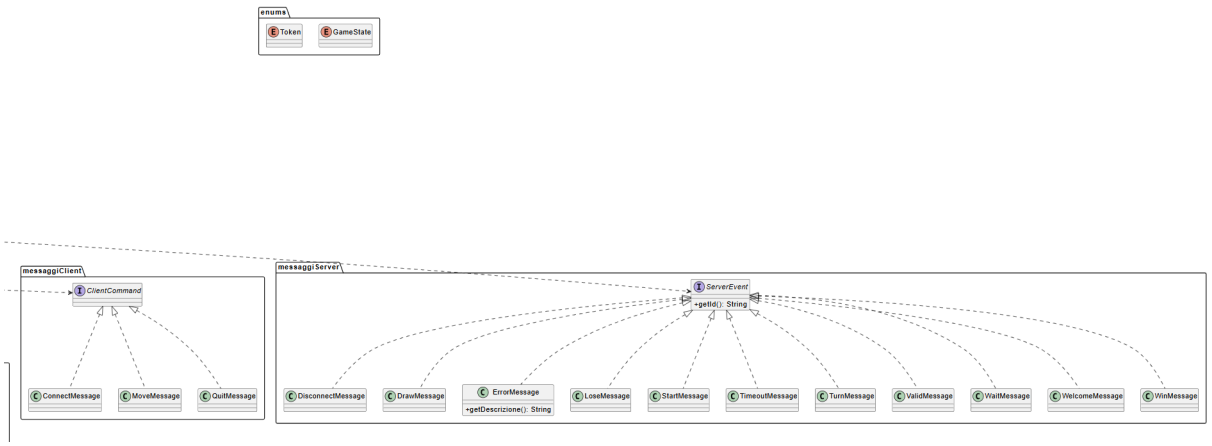

Figura 5: Diagramma Architettura del Server

6 Architettura del Client

Il client permette l'interazione con l'utente e la comunicazione asincrona con il server.

6.1 Componenti del Client

Componente	**Tipo**	**Responsabilità**
Client	Classe	Avvia connessione, ciclo di gioco, invia comandi, gestisce eventi.
ServerListener	Runnable	Legge dal socket e inserisce i messaggi in coda.
CodaCircolare	Classe	Buffer *put/preleva* sincronizzato tra thread.
MessageParser	Classe	Stringa → `ServerEvent`.
MessageFormatter	Classe	`ClientCommand` → stringa.
ClientCommand	Interfaccia	Contratto dei comandi (CONNECT, MOVE, QUIT).
ConnectMessage	Classe	Dati di connessione (nome, età).
MoveMessage	Classe	Dati mossa (colonna).
QuitMessage	Classe	Richiesta di abbandono.
ServerEvent	Interfaccia	Contratto degli eventi con `getId()`/`action()`.
Messaggi server concreti	Classi	*Welcome/Wait/Start/Turn/Valid/Error/Win/Lose/Draw/Timeout/*
ClientCommandType	Enum	Tipi di comando (CONNECT/MOVE/QUIT).
ServerEventType	Enum	Tipi di evento (WELCOME/.../DISCONNECT).

6.2 Gestione del Multithreading Lato Client

Il client utilizza due flussi di esecuzione:

- Thread principale per input utente.
- Thread ServerListener per ascolto continuo.

Questa struttura evita il blocco dell'interfaccia durante l'attesa delle risposte.

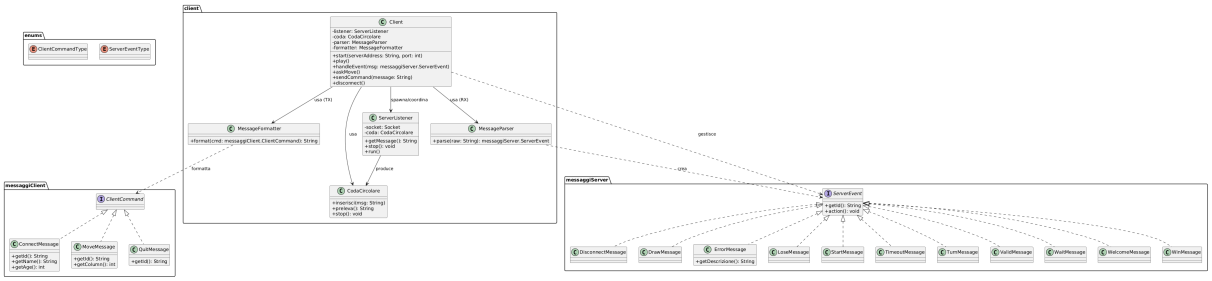

Figura 6: Diagramma Architettura del Server

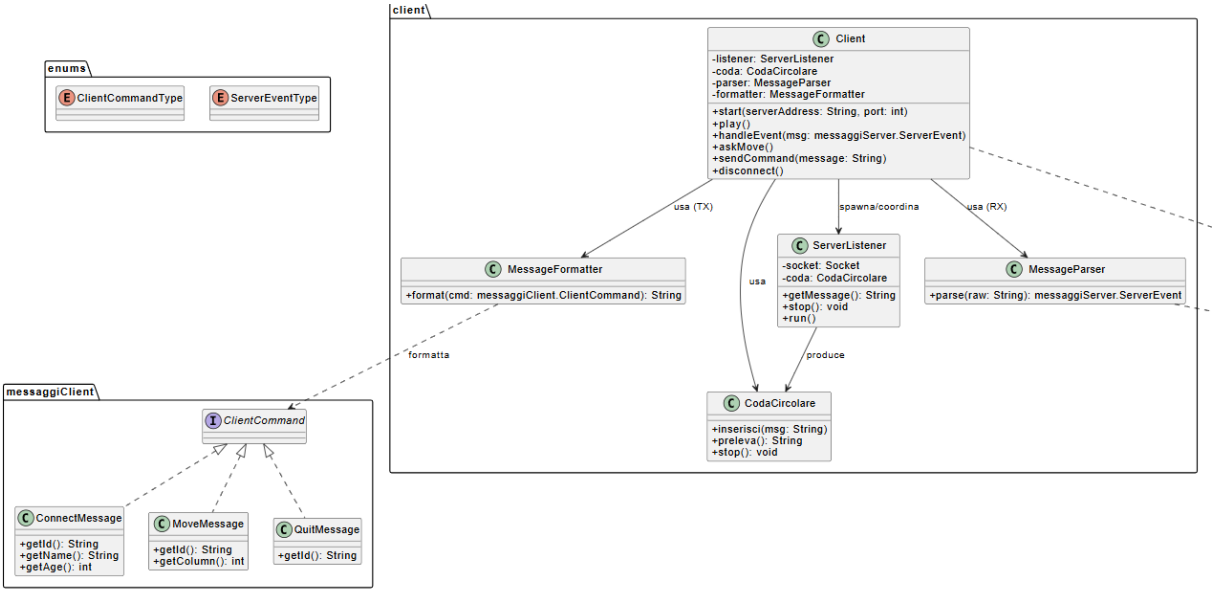

Figura 7: Diagramma Architettura del Server

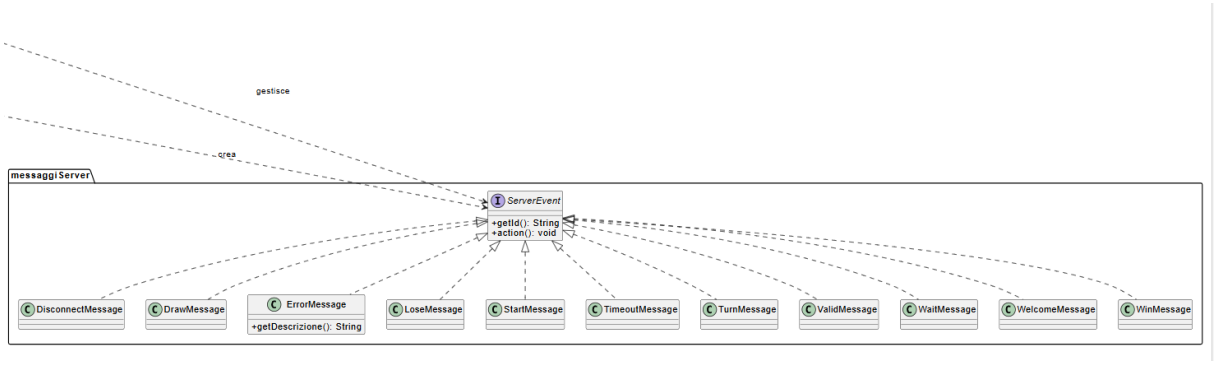

Figura 8: Diagramma Architettura del Server

7 Conformità ai Requisiti

Requisito	Implementazione

Server Multithread	ClientHandler dedicato per ogni client
N Partite Simultanee	GameRoom indipendente per ogni partita
Protocollo Progettato	Messaggi formalizzati e validati
Gestione Concorrenza	Metodi synchronized e isolamento GameRoom

8 Conclusioni

Il progetto ha permesso quindi, la progettazione di un sistema Client-Server multithread per il gioco forza quattro. L'uso di un'architettura centralizzata con un server autoritativo garantisce:

- Coerenza dello stato di gioco
- Validazione corretta delle mosse
- Gestione concorrente delle partite
- Isolamento delle sessioni

La progettazione risulta completa, coerente con i requisiti richiesti e pronta per la fase di implementazione e validazione funzionale.

9 Strumenti utilizzati per i class diagram e sequence diagram

Mermaid.live
PlantUML

Link di GitHub: Repository GitHub